

CAHIER DES CHARGES — RÉVISION TECHNIQUE

BLANCHIX

Plateforme web/PWA de blanchisserie à domicile — spécifications fonctionnelles et techniques détaillées

Version 2.0 — Document de référence contractuelle et technique

Stack : Next.js · PWA · Axios · TanStack React Query · Node.js · MongoDB · Mongoose · NextAuth

Août 2026

Sommaire

Sommaire	2
1. Présentation du projet	6
1.1 Contexte	6
1.2 Objectifs du projet	6
1.3 Utilisateurs cibles	6
1.4 Ce que Blanchix n'est pas	6
2. Périmètre fonctionnel	7
2.1 Vue d'ensemble des modules	7
2.2 Page d'accueil	7
2.3 Comment ça fonctionne ?	7
2.4 Parcours de commande — 5 étapes	8
Étape 1 — Informations du client	8
Étape 2 — Adresse et créneau de collecte	8
Étape 3 — Panier de vêtements	8
Étape 3bis — Article personnalisé (hors catalogue)	9
Étape 3ter — Notes et instructions	10
Étape 4 — Choix du délai et créneau de livraison	10
Étape 5 — Paiement	10
2.5 Tableau de bord administrateur — aperçu	10
2.6 Programme de fidélité — règles métier	11
2.7 Abonnements Basic / Silver / Gold — statut : fonctionnalité V2	11
3. Cycle de vie d'une commande	11
3.1 Diagramme des statuts — parcours nominal	12
3.2 Statuts exceptionnels	12
3.3 Table des transitions et responsabilités	12
3.4 Historique des statuts (statusHistory)	13
4. Architecture technique	14
4.1 Stack technique retenue	14
4.2 Architecture applicative	15
4.3 Justification des choix technologiques	15
5. Modèle de données (MongoDB)	16
Collection : users	16
Collection : orders	16
Collection : catalog_items	17
Collection : payments	18
Collection : incidents	18

Collection : notifications	19
Collection : delivery_slots (créneaux)	19
Collection : audit_logs	20
Collection : subscriptions	20
Collection : loyalty_transactions	20
6. Paiement — module renforcé.....	20
6.1 Principe fondamental.....	20
6.2 Workflow de paiement	20
6.3 Idempotence et gestion des doublons	21
6.4 Gestion des cas d'échec	21
6.5 Comparatif des solutions de paiement	21
6.6 Recalcul des montants côté serveur.....	22
7. Authentification & rôles.....	22
7.1 Authentification (NextAuth)	22
7.2 Rôles et permissions	23
Matrice de permissions indicative (au-delà du MVP)	23
8. Intégrations tierces	24
8.1 Intégrations actives dès le MVP.....	24
8.2 Intégration prévue en V2	24
9. Logistique, créneaux & livraison.....	24
9.1 Créneaux de collecte	24
9.2 Créneaux de livraison	25
9.3 Configuration des créneaux par l'administrateur	25
9.4 Modèle logistique de la commande	25
9.5 Assignation d'un livreur	25
10. Gestion des incidents	25
10.1 Types d'incidents.....	25
10.2 Workflow de traitement d'un incident.....	26
10.3 Modèle de données	26
10.4 Notification liée à un incident	26
11. Notifications	26
11.1 Événements déclencheurs et canaux par phase	26
11.2 États d'envoi et retry.....	27
12. Tableau de bord administrateur.....	27
12.1 Indicateurs clés (KPIs)	27
12.2 Gestion des commandes.....	28
12.3 Gestion du catalogue	28
12.4 Gestion des créneaux et des livreurs.....	28
12.5 Gestion des incidents	28

12.6 Journal d'audit.....	28
13. Audit & traçabilité.....	28
13.1 Principe.....	28
13.2 Actions concernées	29
13.3 Contenu enregistré.....	29
14. Sécurité.....	29
14.1 Validation et intégrité des données	29
14.2 Protection des accès	29
14.3 Sécurité des paiements.....	29
14.4 Gestion des secrets	30
14.5 Journalisation et supervision.....	30
14.6 Protection des données personnelles	30
15. Hébergement, infrastructure & sauvegardes.....	30
15.1 Offre retenue — Contabo Cloud VPS 6.....	30
15.2 Répartition des services sur le VPS.....	31
15.3 Stratégie de sauvegarde	31
15.4 Budget d'hébergement indicatif (hors développement)	31
16. Exigences non fonctionnelles.....	32
16.1 Mobile-first & expérience applicative native.....	32
16.2 Fonctionnement hors ligne — périmètre précis	32
16.3 Performance.....	33
16.4 Compatibilité.....	33
16.5 Protection des données	33
17. Gestion des erreurs	33
18. Tests & qualité.....	34
18.1 Tests unitaires	34
18.2 Tests API	34
18.3 Tests de bout en bout (E2E).....	34
19. Gestion de projet	35
19.1 Objectif du MVP	35
19.2 Contenu du MVP	35
19.3 Contenu de la V1	36
19.4 Contenu de la V2	36
19.5 Livrables attendus	36
19.6 Critères d'acceptation	36
Parcours nominal	36
Cas d'échec et cas limites.....	37
20. Tarification & budget du projet	38
20.1 Méthodologie de chiffrage	38

20.2 Coût de développement par module (MVP)	38
20.3 Coût de développement des évolutions V1 et V2.....	39
20.4 Coûts récurrents mensuels	39
20.5 Frais de transaction (variables selon le volume de commandes).....	39
20.6 Maintenance & support post-lancement	40
20.7 Récapitulatif budgétaire global	40
21. Annexes	40
21.1 Arborescence Next.js indicative	40
21.2 Glossaire	41
21.3 Référentiel des questions clés	41

1. Présentation du projet

1.1 Contexte

Blanchix est un service de blanchisserie 100% digitalisé destiné au marché camerounais (Douala). Le client ne se déplace jamais : il commande en ligne, un coursier collecte son linge à domicile, l'équipe le lave et le repasse en atelier, puis le livre à l'adresse indiquée. L'ensemble du parcours — commande, paiement, suivi, confirmation — est piloté depuis une plateforme web installable comme une application mobile (PWA).

Ce document constitue la version de référence du cahier des charges. Il conserve l'intégralité du périmètre fonctionnel déjà validé (commande en 5 étapes, catalogue dynamique, paiement Mobile Money/carte, suivi, dashboard, fidélité, abonnements, PWA mobile-first) et l'enrichit des workflows, modèles de données, règles métier et exigences techniques nécessaires à une implémentation directe par une équipe de développement.

1.2 Objectifs du projet

- Permettre à un client de commander un service de pressing en moins de 3 minutes, sans appel téléphonique ni déplacement.
- Automatiser le calcul du prix, la génération du reçu et la confirmation (compte et commande) par email au MVP, avec évolution vers WhatsApp en V2.
- Offrir à l'équipe Blanchix un tableau de bord temps réel pour piloter les collectes, les livraisons, les incidents et le chiffre d'affaires.
- Garantir l'intégrité de chaque commande et de chaque paiement grâce à un cycle de vie de statuts strict, tracé et audité.
- Fidéliser la clientèle via un programme de points et, à terme, des formules d'abonnement.
- Garantir une expérience rapide, fiable et proche d'une application native, même en cas de connexion internet instable (PWA mobile-first).

1.3 Utilisateurs cibles

Profil	Besoin principal	Point d'accès
Client particulier	Commander une collecte de linge, suivre sa commande, payer en ligne	Site web / PWA installée sur téléphone
Administrateur Blanchix	Superviser les commandes, la logistique, les incidents, le chiffre d'affaires	Tableau de bord back-office (web)
Coursier / livreur	Consulter les collectes et livraisons qui lui sont assignées (interface dédiée en V2 ; assignation manuelle par l'administrateur au MVP)	Assignation admin (MVP) puis interface mobile simplifiée (V2)

1.4 Ce que Blanchix n'est pas

Pour préserver la cohérence du produit et la faisabilité du MVP, ce cahier des charges ne transforme pas Blanchix en marketplace multi-prestataires, en ERP de gestion d'entreprise généraliste, ni en plateforme logistique tierce. Blanchix reste un service de blanchisserie piloté par une équipe unique : collecte, traitement en atelier, livraison. Toute fonctionnalité hors de ce périmètre (mise en relation de plusieurs pressings, comptabilité complète, gestion de flotte avancée, facturation multi-entités...) est explicitement exclue et devra faire l'objet d'un projet distinct si elle devient nécessaire.

2. Périmètre fonctionnel

2.1 Vue d'ensemble des modules

- Module Accueil & Vitrine — présentation du service, slogan, accès direct à la commande.
- Module « Comment ça fonctionne » — illustration du parcours en 4 étapes.
- Module Commande — parcours en 5 étapes (informations, adresse + créneau, panier, délai + créneau de livraison, paiement).
- Module Paiement — Orange Money, MTN Mobile Money, carte bancaire, avec moteur de vérification serveur (section 6).
- Module Confirmation — reçu PDF généré automatiquement et confirmation par email (compte et commande).
- Module Suivi — cycle de vie complet de la commande avec historique des statuts (section 3).
- Module Créneaux & Logistique — gestion des créneaux de collecte/livraison et assignation d'un livreur (section 9).
- Module Incidents — signalement et traitement des anomalies (section 10).
- Module Notifications — email (MVP), push (V1), WhatsApp (V2) (section 11).
- Module Fidélité — points Blanchix convertibles en réduction, règles détaillées en section 2.6.
- Module Abonnements — formules Basic / Silver / Gold, périmètre détaillé en section 2.7 (V2).
- Module Administration — tableau de bord, gestion du catalogue, des commandes, des créneaux, des incidents, statistiques (section 12).

2.2 Page d'accueil

La page d'accueil affiche le slogan principal « Le linge propre, sans le moindre déplacement. », un bouton d'appel à l'action « Commander maintenant » menant directement au parcours de commande, ainsi que les quatre piliers du service : Collecte à domicile, Lavage professionnel, Repassage, Livraison.

2.3 Comment ça fonctionne ?

Une illustration en 4 étapes explique le parcours au nouveau visiteur :

- ① Le client passe commande en ligne

- ② Blanchix récupère le linge à l'adresse indiquée
- ③ Le linge est lavé puis repassé en atelier
- ④ Le linge propre est livré au client

2.4 Parcours de commande — 5 étapes

Étape 1 — Informations du client

- Nom (obligatoire)
- Téléphone (obligatoire, utilisé pour le suivi et les notifications)
- Email (obligatoire pour le MVP — utilisé pour la confirmation de création de compte, la confirmation de commande et le reçu)
- Numéro WhatsApp (facultatif au MVP ; deviendra le canal de confirmation privilégié en V2, en complément ou à la place de l'email)

Étape 2 — Adresse et créneau de collecte

- Quartier
- Rue
- Localisation GPS via Google Maps (sélection sur carte ou géolocalisation navigateur)
- Créneau de collecte souhaité, choisi parmi les créneaux disponibles publiés par l'administrateur (voir section 9.1) — un créneau complet ou bloqué n'est plus proposé
- Instructions de collecte facultatives (ex. « Livrer/collecter chez le gardien », « Sonner deux fois »)

Étape 3 — Panier de vêtements

Le client compose son panier comme sur un site e-commerce, avec un compteur +/- par article. Le prix total se met à jour instantanément à chaque clic.

Important : la liste des articles proposés et leurs prix ne sont pas figés dans le code. Ils sont entièrement gérés par l'administrateur depuis le tableau de bord (module « Catalogue »), qui peut créer, modifier, désactiver ou supprimer un article et son prix à tout moment, sans intervention technique. Le panier de commande se contente d'afficher dynamiquement le catalogue actif au moment de la commande.

- L'administrateur crée un article avec : nom, prix unitaire (FCFA), catégorie (facultatif), statut actif/inactif.
- Toute modification de prix par l'administrateur s'applique immédiatement aux nouvelles commandes ; les commandes déjà passées conservent le prix appliqué au moment de l'achat (prix figé dans la commande, non recalculé rétroactivement).
- Un article désactivé par l'administrateur n'apparaît plus dans le panier côté client mais reste visible dans l'historique des commandes passées.

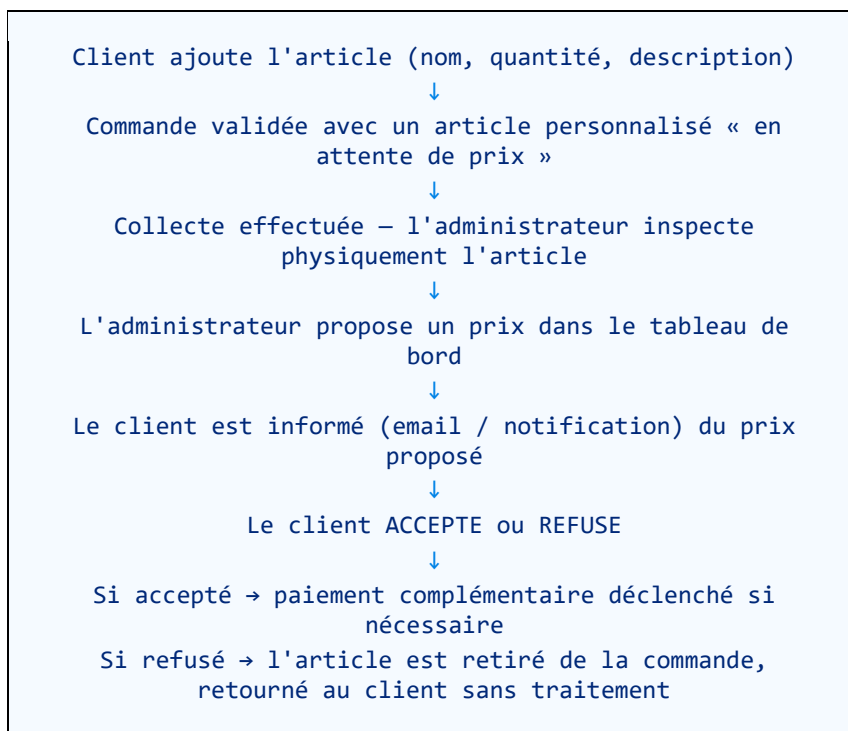
À titre d'exemple, voici un jeu d'articles de démarrage (données d'amorçage) que l'administrateur pourra ensuite modifier librement :

Article (exemple de départ)	Prix unitaire (FCFA)
T-shirt	500
Chemise	700
Pantalon	800
Robe	1 000
Costume	2 000
Chaussures	1 500
Couette	3 500

Ces valeurs ne sont qu'un exemple de contenu initial (« seed ») ; elles ne constituent pas une contrainte de développement et pourront être entièrement redéfinies par l'administrateur avant le lancement.

Étape 3bis — Article personnalisé (hors catalogue)

Le client peut ajouter un article personnalisé, hors catalogue, via « Ajouter un autre article » (nom du vêtement, quantité estimée, description libre — ex. « Chaussure Nike blanche », « Sac à dos », « Rideaux »). Ce type d'article n'a pas de prix prédéfini au moment de la commande : le workflow suivant s'applique.



- Chaque article personnalisé conserve un statut propre : en_attente_prix | prix_propose | accepte | refuse.
- Si le prix proposé diffère du prix estimé indicativement par le client, le total de la commande est recalculé et le complément (positif ou négatif) fait l'objet d'un paiement complémentaire ou d'un avoir, selon le cas.

- Le client ne peut pas être facturé pour un article personnalisé sans avoir explicitement accepté le prix proposé.

Étape 3ter — Notes et instructions

Le client peut ajouter des notes libres à sa commande, distinctes des instructions de collecte/livraison :

- Instructions générales sur le linge (ex. « Ne pas utiliser de parfum », « Attention à cette chemise »)
- Instructions de livraison (ex. « Livrer chez le gardien », « Appeler avant de monter »)

Ces notes sont visibles par l'équipe de traitement et par le livreur, et conservées dans la commande (champ `notesClient`, voir section 5).

Étape 4 — Choix du délai et créneau de livraison

Formule	Délai	Impact sur le prix
Premium	12 h	Majoration la plus élevée
Express	24 h	Majoration modérée
Standard	72 h	Tarif de base

- En complément du délai, le client choisit une date et un créneau de livraison souhaités parmi les créneaux disponibles (voir section 9.2), cohérents avec le délai sélectionné.
- Le client peut confirmer ou modifier l'adresse de livraison si elle diffère de l'adresse de collecte.

Étape 5 — Paiement

- Orange Money
- MTN Mobile Money
- Carte bancaire

Le montant à payer est affiché avant validation. Le workflow de paiement complet (vérification serveur, gestion des callbacks, idempotence) est détaillé en section 6 — le frontend n'est jamais considéré comme la source de vérité de la réussite d'un paiement.

Après paiement confirmé côté serveur, le système génère automatiquement : un reçu PDF téléchargeable, un numéro de commande unique, la date, le nom du client, le montant, la liste des articles et le statut de la commande.

Pour le MVP, la confirmation de commande est envoyée automatiquement par email à l'adresse renseignée à l'étape 1 : un message reprenant le numéro de commande, le nom du client, le montant et le reçu PDF en pièce jointe. Le client reçoit également par email chaque mise à jour importante du statut de sa commande.

Évolution V2 : la confirmation basculera vers WhatsApp (ou le complétera) — un bouton dédié ouvrira automatiquement l'application WhatsApp avec un message pré-rempli contenant le numéro de commande, le nom du client, le montant et le reçu joint, comme décrit en section 8.2.

2.5 Tableau de bord administrateur — aperçu

Après connexion, l'équipe Blanchix accède à une vue de pilotage complète : chiffre d'affaires, commandes, créneaux, incidents, catalogue, statistiques. Le détail exhaustif de ce module est décrit en section 12 pour éviter toute redondance.

2.6 Programme de fidélité — règles métier

Le principe « 100 points = 500 FCFA de réduction » est complété par les règles suivantes, à valider avec le client avant développement :

Règle	Proposition par défaut
Points gagnés par commande	1 point pour chaque 100 FCFA dépensés (hors frais de livraison), arrondi à l'entier inférieur
Moment du crédit des points	À la livraison confirmée de la commande (statut LIVREE), non à la commande ni au paiement
Expiration des points	12 mois après leur date de crédit, à titre indicatif
Montant minimum pour utiliser les points	100 points minimum (soit 500 FCFA de réduction) par utilisation
Plafond de réduction par commande	30% du montant de la commande au maximum, à titre indicatif
Remboursement d'une commande	Les points gagnés sur cette commande sont annulés ; les points utilisés en réduction sont recrédités au client

Ces valeurs par défaut doivent être validées avec Blanchix ; elles servent de base de discussion et n'engagent pas le développement tant qu'elles ne sont pas confirmées.

Toute opération sur les points (gain, utilisation, expiration, annulation) est enregistrée dans la collection `loyalty_transactions` (voir section 5), garantissant la traçabilité complète du solde affiché sur le compte client.

2.7 Abonnements Basic / Silver / Gold — statut : fonctionnalité V2

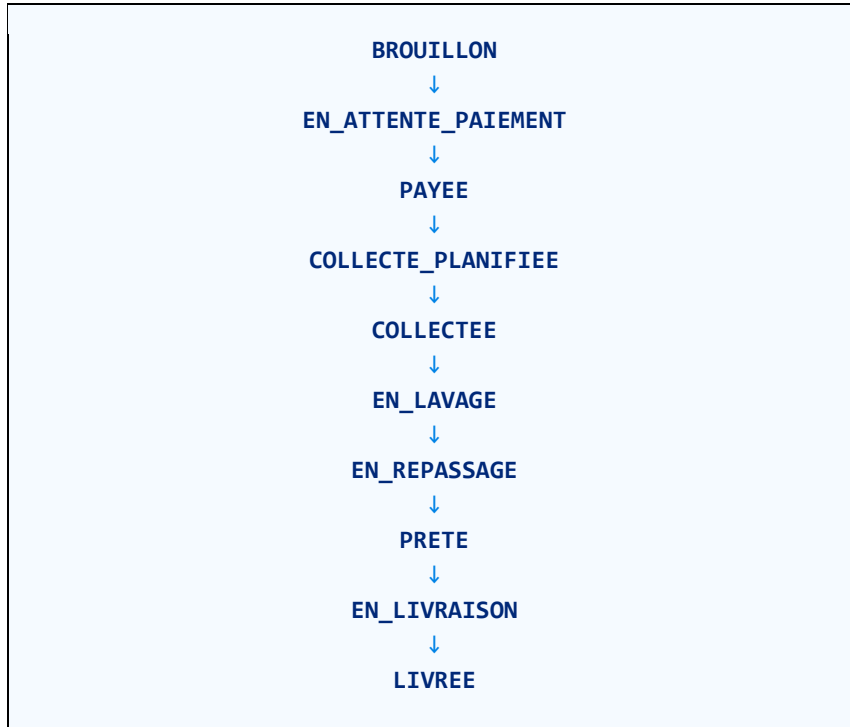
Les formules d'abonnement sont mentionnées dans la vision produit mais leurs règles métier ne sont pas encore validées. Conformément au principe de ne pas développer une fonctionnalité dont les règles ne sont pas définies, les abonnements sont explicitement déplacés en V2. Avant leur développement, le cahier des charges devra préciser pour chaque formule :

- Prix et durée de l'abonnement
- Réduction ou avantage tarifaire accordé
- Avantages non tarifaires (ex. créneau prioritaire, plafond d'articles inclus)
- Conditions d'éligibilité et limites d'usage
- Modalités de renouvellement (automatique ou manuel)
- Modalités d'annulation et de remboursement au prorata

3. Cycle de vie d'une commande

Cette section définit précisément les statuts possibles d'une commande, les transitions autorisées, les responsabilités de chaque changement et l'historique conservé. Elle constitue la référence unique du comportement attendu du système de suivi de commande.

3.1 Diagramme des statuts — parcours nominal



3.2 Statuts exceptionnels

Statut	Déclenché depuis	Description
PAIEMENT_ECHOUE	EN_ATTENTE_PAIEMENT	Le paiement a été refusé, a expiré ou a échoué côté opérateur
ANNULEE	BROUILLON, EN_ATTENTE_PAIEMENT, PAYEE, COLLECTE_PLANIFIEE	Annulation par le client ou l'administrateur avant collecte
COLLECTE_ECHOUEE	COLLECTE_PLANIFIEE	Client absent, adresse incorrecte ou collecte impossible — génère un incident (section 10)
INCIDENT	Tout statut entre COLLECTEE et EN_LIVRAISON	Un incident bloquant a été signalé sur la commande (vêtement endommagé, manquant, etc.)
REMBOURSEE	PAYEE et statuts suivants	Le paiement a été remboursé totalement ou partiellement au client

3.3 Table des transitions et responsabilités

Transition	Déclenchée par	Notification envoyée	Réversible
BROUILLON → EN_ATTENTE_PAIEMENT	Client (validation du panier)	Aucune	Oui (retour au panier)
EN_ATTENTE_PAIEMENT → PAYEE	Backend (callback opérateur validé)	Email de confirmation de commande	Non
EN_ATTENTE_PAIEMENT → PAIEMENT_ECHOUÉ	Backend (callback négatif ou expiration)	Email d'échec de paiement avec lien pour réessayer	Oui (nouvelle tentative de paiement)
PAYEE → COLLECTE_PLANIFIEE	Backend (automatique, dès confirmation créneau)	Email récapitulatif du créneau de collecte	Oui (reprogrammation par l'administrateur)
COLLECTE_PLANIFIEE → COLLECTEE	Administrateur / livreur	Email « linge collecté »	Non
COLLECTE_PLANIFIEE → COLLECTE_ECHOUÉE	Administrateur / livreur	Email + création d'un incident	Oui (reprogrammation)
COLLECTEE → EN_LAVAGE	Administrateur (atelier)	Email « lavage en cours »	Non
EN_LAVAGE → EN_REPASSAGE	Administrateur (atelier)	Email « repassage en cours »	Non
EN_REPASSAGE → PRETE	Administrateur (atelier)	Email « commande prête »	Non
PRETE → EN_LIVRAISON	Administrateur / livreur	Email « en cours de livraison »	Non
EN_LIVRAISON → LIVREE	Administrateur / livreur (confirmation de remise)	Email « commande livrée » + crédit des points de fidélité	Non
Tout statut → ANNULEE	Client (avant collecte) ou administrateur	Email d'annulation	Non
Tout statut → INCIDENT	Administrateur / livreur / client (signalement)	Email + création d'un incident (section 10)	Oui (retour au statut précédent après résolution)
PAYEE et suivants → REMBOURSEE	Administrateur (validation manuelle)	Email de remboursement	Non

Le canal « Email » ci-dessus correspond au MVP ; les canaux push (V1) et WhatsApp (V2) s'ajoutent selon la matrice de la section 11.

3.4 Historique des statuts (statusHistory)

Une commande ne se limite pas à un champ statut unique : elle conserve l'intégralité de son historique, permettant de reconstituer à tout moment le déroulé complet du traitement.

```

statusHistory: [
  { status, date, user, commentaire },
  { status, date, user, commentaire },
  ...
]
```

Cet historique permet de répondre à tout moment aux questions suivantes : quand la commande a été créée, quand le paiement a été confirmé, quand le linge a été collecté, quand le lavage puis le repassage ont commencé, quand la commande était prête, quand elle a été livrée, et qui a effectué chaque changement (client, administrateur nommé, ou système automatique).

- Chaque entrée de statusHistory est immuable une fois créée (append-only) : aucune modification ni suppression rétroactive.
- Le champ user référence soit un compte administrateur, soit la valeur système pour les transitions automatiques (ex. callback de paiement).
- Les changements de statut effectués par un administrateur alimentent également le journal d'audit global (section 13).

4. Architecture technique

4.1 Stack technique retenue

Couche	Technologie	Rôle
Frontend	Next.js (App Router)	Rendu SSR/SSG des pages publiques, routing, API routes légères
Frontend	PWA (manifest + service worker)	Installation sur écran d'accueil, cache offline, notifications push
Frontend	Axios	Client HTTP pour les appels à l'API backend
Frontend	TanStack React Query	Cache, synchronisation et invalidation des données serveur (commandes, panier, statuts)
Frontend	Tailwind CSS	Mise en œuvre de la charte graphique (recommandé, à valider)
Frontend	Composants de navigation mobile (tab bar, bottom sheet)	Reproduire les patterns natifs d'application mobile (navigation basse, feuilles modales, gestes)
Frontend	Web Push API	Notifications push envoyées au client et à l'administrateur, au même titre qu'une application native
Backend	Node.js	Environnement d'exécution du serveur applicatif
Backend	Next.js Route Handlers / API	Exposition des endpoints REST consommés par le frontend
Backend	MongoDB + Mongoose	Base de données documentaire (commandes, utilisateurs, catalogue, paiements)
Backend	NextAuth.js	Authentification client et administrateur (sessions, providers, rôles)
Backend	Zod (ou équivalent)	Validation stricte des schémas de données côté serveur, jamais côté frontend seul

Couche	Technologie	Rôle
Backend	Middleware de rate limiting	Protection contre les requêtes abusives (ex. force brute OTP, spam de commandes)
Intégrations	API Google Maps / Places	Sélection d'adresse et géolocalisation GPS
Intégrations	API Orange Money / MTN MoMo / passerelle carte	Paieement mobile money et carte — voir comparatif section 6.5
Intégrations	Service d'envoi d'email (ex. Resend, SendGrid, Nodemailer)	Confirmation de compte, de commande, de statut — canal principal au MVP
Intégrations	WhatsApp (lien wa.me / WhatsApp Business API)	Confirmation de compte et de commande — prévu en V2
Intégrations	Génération PDF (ex. pdf-lib / Puppeteer)	Génération du reçu de commande

4.2 Architecture applicative

L'application est construite en Next.js full-stack : les pages publiques (accueil, commande, suivi) sont rendues côté serveur pour la performance et le référencement, tandis que le tableau de bord administrateur fonctionne comme une application cliente enrichie, alimentée par React Query. Les échanges entre le frontend et le backend transitent par des endpoints REST exposés via les Route Handlers de Next.js et appelés côté client avec Axios.

- Client (navigateur / PWA installée) → Axios + React Query → API Next.js (Route Handlers)
- API Next.js → Mongoose → MongoDB (persistance)
- API Next.js → NextAuth → gestion des sessions et des rôles (client / administrateur)
- API Next.js → Services externes (Maps, Mobile Money, carte bancaire, email transactionnel, génération PDF ; WhatsApp en V2)
- Service Worker (PWA) → mise en cache des ressources statiques et des dernières données consultées, file d'attente des actions non critiques en cas de perte de connexion (jamais le paiement, voir section 16.2)
- Toute logique de calcul de prix, de vérification de paiement et de changement de statut réside exclusivement côté backend — le frontend affiche, il ne décide jamais seul.

4.3 Justification des choix technologiques

- Next.js : un seul projet couvre le rendu des pages publiques, les API internes et l'espace administrateur, ce qui simplifie le déploiement et la maintenance.
- PWA : indispensable sur un marché où l'installation d'une application native (App Store / Play Store) est un frein ; la PWA offre une expérience proche du natif (icône, plein écran, notifications) sans passer par les stores.
- Axios + TanStack React Query : Axios structure les appels HTTP (intercepteurs, gestion d'erreurs), React Query prend en charge le cache, le rafraîchissement automatique des statuts de commande et la synchronisation optimiste du panier.

- MongoDB : modèle documentaire adapté à des commandes composées d'articles variables (paniers hétérogènes) et à un historique de statuts imbriqué, sans schéma rigide de type SQL.
- NextAuth : intégration native à Next.js, gestion simplifiée des sessions et des rôles (client / administrateur), compatible avec une authentification par téléphone/OTP ou identifiants classiques.
- Zod : garantit que toute donnée reçue par l'API (formulaire, panier, webhook de paiement) est strictement conforme au schéma attendu avant tout traitement métier.

5. Modèle de données (MongoDB)

Le modèle ci-dessous liste les collections principales et leurs champs clés, enrichi pour couvrir le cycle de vie complet des commandes, les paiements, la logistique, les incidents, les notifications et l'audit. Il sera affiné lors du cadrage technique (index, relations, versionnement).

Collection : users

Champ	Type	Description
_id	ObjectId	Identifiant unique
nom	String	Nom du client
telephone	String	Numéro de téléphone, identifiant de connexion
email	String	Email — obligatoire au MVP, utilisé pour la confirmation de création de compte, la confirmation de commande et le reçu
whatsapp	String	Numéro WhatsApp — facultatif au MVP, deviendra le canal de confirmation (compte et commande) en V2
role	String	client admin operateur livreur super_admin (rôles étendus décrits en section 7.2)
points	Number	Points de fidélité cumulés
abonnement	String	aucun basic silver gold (V2)
adresses	Array	Liste des adresses enregistrées
compteConfirme	Object	Canal utilisé (email au MVP, whatsapp en V2), statut d'envoi, date d'envoi du message de bienvenue
createdAt / updatedAt	Date	Horodatage

Collection : orders

Le document de commande conserve un véritable instantané (snapshot) des informations importantes au moment de l'achat : articles avec prix figé, adresses, créneaux, montants détaillés et informations de paiement. Rien n'est recalculé rétroactivement depuis le catalogue ou les tarifs courants.

Champ	Type	Description
_id / numero	ObjectId / String	Identifiant unique et numéro lisible, ex. BLX-20260807001
client	ObjectId (ref users)	Client ayant passé la commande
statut	String	Statut courant — voir énumération complète section 3.1 / 3.2
statusHistory	Array	Historique complet des statuts : { status, date, user, commentaire } — voir section 3.4
articles	Array	Articles du catalogue : { catalogItemId, nom, prixUnitaire, quantite, description }, prix figé à la commande
articlesPersonnalisés	Array	Articles hors catalogue : { nom, quantiteEstimee, description, prixPropose, statutValidation, dateProposition, dateReponse } — voir section 2.4 (étape 3bis)
adresseCollecte / adresseLivraison	Object	Quartier, rue, coordonnées GPS, instructions spécifiques
creneauCollecte / creneauLivraison	Object (ref delivery_slots)	Date et plage horaire retenues — voir section 9
livreurCollecte / livreurLivraison	ObjectId (ref users, rôle livreur)	Livreur assigné — assignation manuelle par l'administrateur au MVP
delai	String	premium express standard
notesClient	String	Instructions libres du client sur le linge ou la livraison — voir section 2.4 (étape 3ter)
sousTotal / fraisLivraison / majorationDelai / reduction / total	Number	Détail complet du calcul du prix, recalculé exclusivement côté serveur
pointsGagnes / pointsUtilises	Number	Mouvements de fidélité liés à cette commande, en miroir de loyalty_transactions
paiement	Object	Méthode, statut, référence de transaction (ref payments)
confirmation	Object	Canal utilisé (email au MVP, whatsapp en V2), statut d'envoi, date d'envoi
recuPdfUrl	String	Lien vers le reçu généré
createdAt / updatedAt	Date	Horodatage

Collection : catalog_items

Référentiel dynamique des articles proposés dans le panier, entièrement piloté par l'administrateur (création, modification du prix, activation/désactivation, suppression). Aucun article ni prix n'est codé en dur dans l'application.

Champ	Type	Description
_id	ObjectId	Identifiant unique de l'article

Champ	Type	Description
nom	String	Nom de l'article (ex. T-shirt, Chemise, Costume)
prixUnitaire	Number	Prix en FCFA, modifiable à tout moment par l'administrateur
categorie	String	Regroupement facultatif (ex. Vêtements, Linge de maison)
actif	Boolean	true si l'article est proposé dans le panier client, false s'il est désactivé
ordreAffichage	Number	Position d'affichage dans la liste du panier (facultatif)
creePar / modifiePar	ObjectId (ref users)	Administrateur ayant créé / modifié l'article
createdAt / updatedAt	Date	Horodatage, utile pour l'historique des changements de prix

Fonctionnalités attendues côté administrateur pour ce module : liste des articles avec recherche/tri, formulaire de création/édition (nom, prix, catégorie, statut actif), activation/désactivation en un clic, et journal des modifications de prix pour la traçabilité (voir section 13).

Collection : payments

Journal exhaustif des transactions de paiement, conçu pour garantir l'idempotence et la traçabilité complète — voir le workflow détaillé en section 6.

Champ	Type	Description
_id	ObjectId	Identifiant unique de la transaction
order	ObjectId (ref orders)	Commande associée
methode	String	orange_money mtn_momo carte
montant / devise	Number / String	Montant exact et devise (FCFA)
statut	String	initie en_attente reussi echoue expire rembourse
referenceExterne	String	Référence de transaction fournie par l'opérateur / la passerelle
idempotencyKey	String	Clé unique générée à l'initiation, empêchant tout double traitement d'un même paiement
callbackRecuLe	Array de Date	Horodatage de chaque callback/webhook reçu, y compris les doublons, pour audit
tentatives	Number	Nombre de tentatives de paiement pour cette commande
logsBruts	Object	Charge utile brute du dernier webhook reçu, à des fins de diagnostic
createdAt / updatedAt	Date	Horodatage

Collection : incidents

Champ	Type	Description
_id	ObjectId	Identifiant unique
commande	ObjectId (ref orders)	Commande concernée
type	String	client_absent adresse_incorrecte vetement_manquant vetement_endommagement vetement_tache probleme_paiement probleme_livraison retard autre
description	String	Description libre de l'incident
createur	ObjectId (ref users)	Personne ayant signalé l'incident (client, administrateur, livreur)
statut	String	ouvert en_cours resolu clos
resolution	String	Description de la résolution apportée
historique	Array	Suivi des changements de statut de l'incident, sur le même principe que statusHistory
createdAt / updatedAt	Date	Horodatage

Collection : notifications

Champ	Type	Description
_id	ObjectId	Identifiant unique
destinataire	ObjectId (ref users)	Client ou administrateur destinataire
evenement	String	Événement déclencheur — voir la matrice complète section 11.1
canal	String	email push whatsapp (selon la phase MVP/V1/V2)
statut	String	pending sent failed
tentatives	Number	Nombre de tentatives d'envoi, avec logique de retry
dateEnvoi	Date	Date d'envoi effective
contenuResume	String	Résumé du contenu envoyé, à des fins d'audit et de support client

Collection : delivery_slots (créneaux)

Champ	Type	Description
_id	ObjectId	Identifiant unique
date	Date	Date du créneau
plageHoraire	String	Ex. 08h00–10h00, 10h00–12h00, 14h00–16h00, 16h00–18h00
type	String	collecte livraison
capaciteMax	Number	Nombre maximum de commandes acceptées sur ce créneau
reserves	Number	Nombre de commandes déjà positionnées sur ce créneau

Champ	Type	Description
statut	String	ouvert complet bloqué (configurable par l'administrateur)

Collection : audit_logs

Champ	Type	Description
_id	ObjectId	Identifiant unique
utilisateur	ObjectId (ref users)	Auteur de l'action (compte administrateur)
action	String	Nature de l'action (ex. modification_prix, changement_statut, remboursement)
cible	Object	Type et identifiant de l'entité modifiée (commande, article, utilisateur, abonnement...)
ancienneValeur / nouvelleValeur	Mixed	État avant/après de la donnée modifiée
adresseIP	String	Adresse IP à l'origine de l'action
date	Date	Horodatage de l'action

Collection : subscriptions

Historique des abonnements souscrits par les clients (formule, date de début/fin, statut, avantages associés) — fonctionnalité V2, champs à finaliser selon les règles métier validées (section 2.7).

Collection : loyalty_transactions

Historique de tous les mouvements de points (gain, utilisation, expiration, annulation lors d'un remboursement) permettant la traçabilité complète du solde affiché dans users.points, conformément aux règles de la section 2.6.

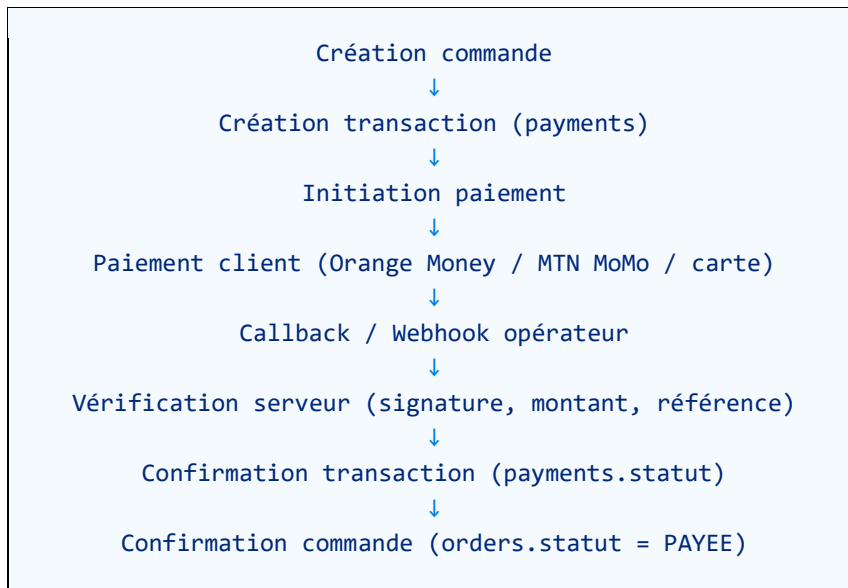
6. Paiement — module renforcé

6.1 Principe fondamental

Important — le frontend ne doit jamais être considéré comme la source de vérité concernant la réussite d'un paiement. Seule une confirmation vérifiée côté serveur (callback/webhook validé et contrôlé) peut faire passer une commande au statut PAYEE.

Le rôle du frontend se limite à initier le paiement, afficher son statut (en interrogeant le backend) et informer l'utilisateur. Toute décision — succès, échec, montant, double tentative — est tranchée exclusivement côté serveur.

6.2 Workflow de paiement



6.3 Idempotence et gestion des doublons

Le backend doit garantir qu'aucune opération de paiement ne peut être traitée deux fois, quelle que soit son origine :

- Une idempotencyKey est générée à l'initiation du paiement et associée à la transaction ; toute requête ou callback portant la même clé est reconnue et ignorée si déjà traitée.
- Un double clic du client sur « Payer » ne doit jamais créer deux transactions distinctes pour la même commande en attente de paiement.
- Un callback reçu plusieurs fois pour la même transaction (retry réseau côté opérateur) ne doit déclencher qu'une seule mise à jour de statut ; les réceptions suivantes sont journalisées (callbackRecuLe) mais sans effet supplémentaire.

6.4 Gestion des cas d'échec

Cas	Comportement attendu
Paielement échoué (refus opérateur)	Transaction marquée echoue, commande repasse en PAIEMENT_ECHOUE, email envoyé avec lien pour réessayer
Paielement expiré (délai dépassé sans callback)	Transaction marquée expire après un délai défini (ex. 15 minutes), commande repasse en PAIEMENT_ECHOUE
Callback reçu plusieurs fois	Traité une seule fois grâce à l'idempotencyKey ; les réceptions suivantes sont journalisées sans effet
Double clic sur « Payer »	Le bouton est désactivé après le premier clic ; le backend refuse toute seconde initiation tant qu'une transaction est en_attente sur la même commande
Paielement réussi mais commande déjà annulée	Le paiement est accepté mais la commande reste ANNULEE ; un remboursement est déclenché manuellement par l'administrateur

6.5 Comparatif des solutions de paiement

Le choix définitif du ou des prestataires doit être arrêté avant le début de l'implémentation. Le tableau ci-dessous structure la comparaison ; il ne remplace pas la validation contractuelle finale.

Critère	Stripe	CinetPay	Flutterwave
Disponibilité Cameroun	Cartes internationales uniquement, pas de Mobile Money local natif	Oui — agrège Orange Money, MTN MoMo et cartes en une seule intégration	Oui — couverture large Afrique de l'Ouest/Centrale, cartes et Mobile Money
Frais indicatifs	≈ 2,9% + frais fixe (cartes internationales)	≈ 1 à 3% selon le moyen de paiement	≈ 1,4 à 3,8% selon le moyen de paiement et la devise
Facilité d'intégration	Excellente documentation, SDK Node.js mature	Documentation correcte, intégration REST directe	Documentation correcte, SDK Node.js disponible
Webhooks	Très robustes, signatures HMAC bien documentées	Disponibles, vérification de signature à implémenter avec soin	Disponibles, vérification de signature à implémenter avec soin
Remboursement	Natif et simple via API	Disponible, process manuel ou API selon contrat	Disponible via API
Devise FCFA	Non natif (conversion nécessaire)	Natif FCFA	Natif FCFA (XAF/XOF)
Support local	Support international uniquement	Support régional francophone	Support régional

Lecture indicative : pour un lancement centré sur le Cameroun avec Mobile Money natif en FCFA, CinetPay et Flutterwave sont les candidats les plus pertinents, Stripe restant une option limitée aux paiements par carte internationale. Cette recommandation doit être confirmée avec Blanchix (frais réels négociés, qualité du support, délais d'ouverture de compte marchand) avant tout développement d'intégration.

Concernant les APIs Orange Money et MTN Mobile Money : le choix entre une intégration directe auprès de chaque opérateur et l'utilisation d'une passerelle unique agréant les deux (ex. CinetPay) doit être tranché avant l'implémentation. Une passerelle unique réduit la complexité d'intégration et de maintenance (un seul jeu de webhooks, une seule réconciliation) au prix d'une dépendance à un tiers supplémentaire et de frais légèrement supérieurs à une intégration directe.

6.6 Recalcul des montants côté serveur

Important — le serveur ne doit jamais faire confiance aux montants envoyés par le frontend. Le prix (sous-total, frais de livraison, majoration de délai, réduction, total) est systématiquement recalculé côté backend à partir du catalogue courant, des règles tarifaires et des points de fidélité réellement disponibles, avant toute initiation de paiement.

7. Authentification & rôles

7.1 Authentification (NextAuth)

- Client : connexion par numéro de téléphone avec code OTP (SMS), ou par email si renseigné — un compte est créé automatiquement dès la première commande.

- Confirmation de création de compte — pour le MVP, un email de bienvenue est envoyé automatiquement à l'adresse renseignée dès la création du compte (récapitulant les informations enregistrées et un lien vers l'espace client) ; ce message basculera vers WhatsApp (ou le complètera) à partir de la V2, selon le même principe que la confirmation de commande (section 8.2).
- Administrateur : connexion par identifiants (email + mot de passe) via un provider Credentials NextAuth, avec session sécurisée.
- Gestion de session : NextAuth gère les jetons de session (JWT) et leur renouvellement, exploités par le middleware Next.js pour protéger les routes /admin.

7.2 Rôles et permissions

Le rôle admin unique suffit pour le MVP, mais l'architecture des permissions est conçue dès le départ pour accueillir une hiérarchie de rôles plus fine, sans refonte ultérieure du modèle de données (le champ role de la collection users accepte déjà ces valeurs, voir section 5).

Rôle	Portée prévue
client	Parcours de commande, suivi de ses commandes, historique, points de fidélité, abonnement
super_admin	Accès complet, y compris gestion des comptes administrateurs et des permissions (au-delà du MVP)
admin	Tableau de bord complet : commandes, catalogue, créneaux, incidents, statistiques, clients — rôle unique utilisé au MVP
operateur	Sous-ensemble des droits admin, orienté traitement quotidien (commandes, incidents) sans accès aux paramètres sensibles (au-delà du MVP)
livreur	Liste des collectes/livraisons assignées, mise à jour du statut de commande (interface dédiée en V2)

Matrice de permissions indicative (au-delà du MVP)

Action	SUPER_ADMIN	ADMIN	OPERATEUR	LIVREUR
Gérer le catalogue (créer/modifier les prix)	Oui	Oui	Non	Non
Gérer les commandes (statuts, créneaux)	Oui	Oui	Oui	Statuts logistiques uniquement
Gérer les utilisateurs et les rôles	Oui	Non	Non	Non
Gérer les paiements et remboursements	Oui	Oui	Non	Non
Voir les statistiques	Oui	Oui	Oui	Non
Gérer les livreurs et les assignations	Oui	Oui	Non	Non

Cette matrice n'est pas implémentée au MVP (rôle admin unique) mais dimensionne le modèle de permissions pour éviter une refonte lors de son introduction.

8. Intégrations tierces

8.1 Intégrations actives dès le MVP

Intégration	Usage	Point d'attention
Google Maps / Places API	Sélection d'adresse, géolocalisation GPS à l'étape 2 de la commande	Clé API restreinte par domaine, quota de requêtes à surveiller ; prévoir une saisie manuelle de repli en cas d'erreur (section 22)
Orange Money API / passerelle	Paieement mobile money	Choix définitif à valider — voir comparatif section 6.5
MTN Mobile Money API / passerelle	Paieement mobile money	Choix définitif à valider — voir comparatif section 6.5
Passerelle carte bancaire	Paieement par carte	Choix définitif à valider — voir comparatif section 6.5
Service d'envoi d'email (ex. Resend, SendGrid, Nodemailer + SMTP)	Confirmation de création de compte, confirmation de commande, reçu PDF en pièce jointe, notifications de changement de statut	Domaine d'envoi à authentifier (SPF/DKIM) pour éviter le classement en spam
Génération de PDF	Reçu de commande téléchargeable et joint à l'email de confirmation	Génération côté serveur (ex. Puppeteer ou pdf-lib) déclenchée après paiement validé
Notifications push (PWA)	Alerte immédiate à l'administrateur lors d'une nouvelle commande — voir section 11	Web Push API + service worker, autorisation à demander à l'utilisateur

8.2 Intégration prévue en V2

Intégration	Usage	Point d'attention
WhatsApp (wa.me puis WhatsApp Business API)	Confirmation de création de compte et confirmation de commande avec message pré-rempli et reçu joint, en remplacement ou en complément de l'email	Version initiale V2 : lien wa.me (ouverture manuelle) ; évolution vers WhatsApp Business API pour l'envoi automatique côté serveur

9. Logistique, créneaux & livraison

Même si l'interface dédiée au livreur est planifiée en V2, le modèle métier de la logistique est conçu dès le MVP pour éviter toute refonte ultérieure.

9.1 Créneaux de collecte

Le client choisit un créneau de collecte parmi une liste configurée par l'administrateur, par exemple :

- 08h00 – 10h00

- 10h00 – 12h00
- 14h00 – 16h00
- 16h00 – 18h00

9.2 Créneaux de livraison

De la même manière, le client choisit une date et un créneau de livraison souhaités, cohérents avec le délai retenu (Premium/Express/Standard). Le client peut également préciser sa disponibilité et des instructions particulières pour la livraison (ex. « Livrer chez le gardien »).

9.3 Configuration des créneaux par l'administrateur

- L'administrateur définit les créneaux disponibles (date, plage horaire, type collecte/livraison) et leur capacité maximale.
- Lorsqu'un créneau atteint sa capacité maximale, il passe automatiquement au statut complet et n'est plus proposé au client.
- L'administrateur peut également bloquer manuellement un créneau (statut bloqué), par exemple en cas d'indisponibilité exceptionnelle de l'équipe.

9.4 Modèle logistique de la commande

Chaque commande porte les informations logistiques suivantes (détaillées dans le modèle de données, section 5) :

```

adresseCollecte
adresseLivraison
livreurCollecte
livreurLivraison
creneauCollecte
creneauLivraison

```

9.5 Assignment d'un livreur

- Au MVP, l'assignation d'un livreur à une collecte ou une livraison est effectuée manuellement par l'administrateur depuis le tableau de bord.
- En V2, une interface dédiée permettra au livreur de consulter directement ses collectes et livraisons assignées, et de mettre à jour lui-même le statut correspondant (COLLECTEE, EN_LIVRAISON, LIVREE).

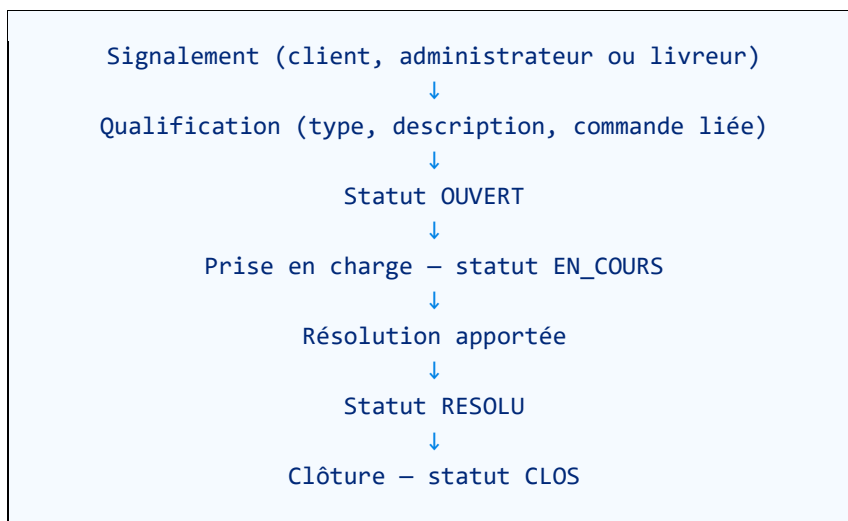
10. Gestion des incidents

10.1 Types d'incidents

- Client absent au moment de la collecte ou de la livraison

- Adresse incorrecte ou introuvable
- Vêtement manquant
- Vêtement endommagé
- Vêtement taché (non traitable)
- Problème de paiement
- Problème de livraison
- Retard
- Autre incident (champ libre)

10.2 Workflow de traitement d'un incident



Un incident peut, selon sa gravité, faire passer la commande liée au statut exceptionnel INCIDENT (section 3.2), suspendant sa progression normale jusqu'à résolution.

10.3 Modèle de données

Le schéma complet de la collection incidents est décrit en section 5. Chaque incident conserve un historique de ses propres changements de statut, sur le même principe que statusHistory pour les commandes.

10.4 Notification liée à un incident

La création d'un incident déclenche automatiquement une notification vers le client concerné (information transparente) et vers l'administrateur (prise en charge rapide), selon la matrice de la section 11.

11. Notifications

11.1 Événements déclencheurs et canaux par phase

Événement	MVP	V1	V2
Création de compte	Email	Email + Push (admin)	+ WhatsApp
Commande créée	Email	Email + Push (admin)	+ WhatsApp
Paielement confirmé	Email	Email + Push	+ WhatsApp
Paielement échoué	Email	Email + Push	+ WhatsApp
Collecte planifiée	Email	Email + Push	+ WhatsApp
Commande collectée	Email	Email + Push	+ WhatsApp
Lavage commencé	Email	Email + Push	+ WhatsApp
Repassement commencé	Email	Email + Push	+ WhatsApp
Commande prête	Email	Email + Push	+ WhatsApp
Commande en livraison	Email	Email + Push	+ WhatsApp
Commande livrée	Email	Email + Push	+ WhatsApp
Incident signalé	Email	Email + Push (admin prioritaire)	+ WhatsApp

11.2 États d'envoi et retry

Chaque notification transite par les états pending → sent ou failed (voir collection notifications, section 5).

- pending : notification créée, envoi en cours de traitement.
- sent : notification délivrée avec succès au fournisseur d'envoi (email, push ou WhatsApp).
- failed : échec d'envoi ; une politique de retry (ex. 3 tentatives avec délai croissant) est appliquée avant abandon définitif et alerte interne.
- Un échec d'envoi de notification n'affecte jamais le statut métier de la commande : la progression du traitement du linge n'est jamais bloquée par un problème d'envoi d'email.

12. Tableau de bord administrateur

Cette section détaille le module d'administration introduit en section 2.5, en cohérence avec les workflows définis dans les sections précédentes.

12.1 Indicateurs clés (KPIs)

- Chiffre d'affaires
- Commandes du jour
- Commandes en attente

- Commandes en cours (collecte → prête)
- Commandes livrées
- Paiements réussis
- Paiements échoués
- Collectes du jour
- Livraisons du jour
- Clients actifs

12.2 Gestion des commandes

L'administrateur dispose d'une vue liste des commandes avec :

- Recherche (par numéro de commande, nom ou téléphone du client)
- Filtres par statut, période, moyen de paiement, client, montant
- Accès au détail complet d'une commande : articles, historique des statuts, paiement, créneaux, incidents liés
- Actions de changement de statut conformes à la table des transitions (section 3.3)

12.3 Gestion du catalogue

Reprend les fonctionnalités déjà décrites en section 2.4 et 5 : création, modification du prix, activation/désactivation, catégories, ordre d'affichage, et désormais un historique consultable des modifications de prix (issu du journal d'audit, section 13).

12.4 Gestion des créneaux et des livreurs

- Création, modification et blocage des créneaux de collecte et de livraison (section 9.3).
- Vue des collectes et livraisons du jour, avec assignation manuelle d'un livreur à chaque commande.

12.5 Gestion des incidents

- Liste des incidents ouverts et en cours, avec priorisation par ancienneté ou gravité.
- Formulaire de résolution et de clôture d'un incident (section 10.2).

12.6 Journal d'audit

Un accès en lecture au journal d'audit (section 13) permet de retracer les actions sensibles effectuées sur le catalogue, les commandes, les paiements et les comptes.

13. Audit & traçabilité

13.1 Principe

Toute action administrative sensible est enregistrée dans la collection `audit_logs` (section 5), de manière à pouvoir répondre à tout moment à la question « qui a fait quoi, quand, et avec quelle donnée avant/après ? ».

13.2 Actions concernées

- Modification d'un prix d'article du catalogue
- Changement de statut d'une commande effectué manuellement par un administrateur
- Annulation d'une commande
- Remboursement d'un paiement
- Modification manuelle d'une commande (ex. correction d'adresse, ajout d'un article personnalisé)
- Modification d'un abonnement (V2)
- Création, modification ou désactivation d'un compte administrateur (au-delà du MVP)

13.3 Contenu enregistré

Chaque entrée du journal d'audit conserve : l'utilisateur auteur de l'action, l'action réalisée, la cible concernée, l'ancienne valeur, la nouvelle valeur, la date et l'adresse IP à l'origine de la requête (voir schéma détaillé section 5).

14. Sécurité

14.1 Validation et intégrité des données

- Toute donnée reçue par l'API (formulaires, panier, webhooks de paiement) est validée côté serveur avec Zod (ou équivalent), indépendamment de toute validation déjà effectuée côté frontend.

Important — le serveur recalcule systématiquement le prix d'une commande à partir du catalogue courant et des règles tarifaires en vigueur ; aucun montant transmis par le frontend n'est jamais utilisé tel quel pour un paiement.

14.2 Protection des accès

- Rate limiting sur les endpoints sensibles (envoi d'OTP, tentatives de connexion, création de commande) pour limiter les abus et les attaques par force brute.
- Protection des routes administrateur par middleware NextAuth avec contrôle de rôle sur chaque requête.
- Contrôle d'accès strict côté serveur : chaque endpoint vérifie l'autorisation réelle de l'utilisateur, indépendamment de ce qu'affiche l'interface.

14.3 Sécurité des paiements

- Vérification systématique de la signature des webhooks de paiement avant tout traitement (rejet de toute requête non authentifiée comme provenant de l'opérateur).

- Idempotence garantie sur toute opération de paiement (section 6.3).
- Aucune donnée de carte bancaire stockée en base — délégation intégrale à la passerelle de paiement (conformité PCI-DSS).

14.4 Gestion des secrets

- Toutes les clés API, secrets NextAuth et identifiants de connexion aux services tiers sont stockés en variables d'environnement, jamais en dur dans le code.
- Politique de rotation périodique des secrets sensibles (clés API de paiement, secrets de session).

14.5 Journalisation et supervision

- Journalisation structurée des requêtes API, des erreurs et des tentatives de paiement, avec un identifiant de requête permettant de relier les logs entre le frontend, l'API et les services externes (voir aussi section 17).
- Supervision de la disponibilité et de la charge du serveur (VPS) et de la base MongoDB.

14.6 Protection des données personnelles

- Collecte limitée aux données nécessaires au service (nom, téléphone, email, adresse, historique de commandes).
- Politique de confidentialité accessible depuis le site, information claire sur l'usage des données de localisation et des notifications.

15. Hébergement, infrastructure & sauvegardes

L'application (frontend Next.js, API et base MongoDB) est hébergée sur un serveur privé virtuel (VPS) Contabo, offrant un bon rapport performance/coût pour un lancement en Afrique de l'Ouest/Centrale, avec possibilité de montée en charge par la suite.

15.1 Offre retenue — Contabo Cloud VPS 6

Caractéristique	Détail
Processeur	6 vCPU Cores
Mémoire RAM	12 Go
Stockage	200 Go SSD (extensible)
Snapshots	2 snapshots inclus
Réseau	Port 300 Mbit/s
Trafic	Illimité*
Tarif promotionnel	6,00 € / mois pendant les 24 premiers mois

Caractéristique	Détail
Tarif standard	7,50 € / mois à partir du 25 ^e mois

**Trafic soumis à la politique d'usage raisonnable du fournisseur.*

15.2 Répartition des services sur le VPS

- Application Next.js (frontend + API Route Handlers) — conteneur applicatif principal.
- Base de données MongoDB — instance auto-hébergée sur le VPS (alternative : MongoDB Atlas en mode managé si une externalisation est préférée, avec coût additionnel).
- Reverse proxy (Nginx) et certificat SSL (Let's Encrypt, gratuit) pour sécuriser le nom de domaine en HTTPS.
- Stockage des reçus PDF générés.

15.3 Stratégie de sauvegarde

Important — le backup ne doit jamais être stocké uniquement sur le même VPS que la base de données de production — la perte du serveur entraînerait alors la perte simultanée des données et de leur sauvegarde.

La stratégie de sauvegarde combine trois niveaux complémentaires :

Backup MongoDB quotidien (mongodump programmé)
+
Snapshots VPS Contabo (image système complète)
+
Backup externe (stockage distinct, hors du VPS de production)

- Le backup MongoDB quotidien est exporté vers un stockage externe (ex. espace de stockage objet distinct du VPS de production).
- Les snapshots Contabo couvrent une restauration rapide de l'ensemble du serveur en cas d'incident système.
- Une procédure de restauration est documentée et testée périodiquement, afin de garantir qu'une sauvegarde est effectivement exploitable en cas de besoin réel — une sauvegarde jamais testée n'est pas une garantie.

15.4 Budget d'hébergement indicatif (hors développement)

Poste	Coût estimé	Périodicité
VPS Contabo Cloud VPS 6	6,00 € / mois (24 premiers mois), puis 7,50 €/mois	Mensuel
Nom de domaine (.com / .cm)	10 à 20 € / an selon extension	Annuel
Certificat SSL (Let's Encrypt)	0 €	Gratuit, renouvellement automatique

Poste	Coût estimé	Périodicité
Stockage de sauvegarde externe	2 à 5 € / mois selon volume	Mensuel

Ces montants couvrent uniquement l'infrastructure ; les frais des passerelles de paiement et des API tierces sont facturés séparément selon l'usage (voir section 20).

16. Exigences non fonctionnelles

16.1 Mobile-first & expérience applicative native

L'application doit être conçue mobile-first : chaque écran est pensé en priorité pour le smartphone, la version bureau étant une adaptation de l'expérience mobile et non l'inverse. Sur mobile, l'objectif est que le client ait la sensation d'utiliser une véritable application native téléchargée sur un store, et non un simple site web responsive.

- Navigation par tab bar — une barre de navigation basse fixe (icônes + libellés courts, ex. Accueil, Commander, Suivi, Fidélité, Compte), toujours visible, comme sur une application native iOS/Android.
- Composants natifs à l'écran — champs de formulaire, sélecteurs de date/heure, boutons d'action, listes déroulantes et cases à cocher repris avec des styles et interactions proches des composants natifs du système.
- Barre supérieure applicative — en-tête compact avec titre d'écran, bouton retour natif, et actions contextuelles (ex. icône de notification).
- Transitions et gestes — animations de transition entre écrans, geste de retour par balayage (« swipe back »), tirer-pour-rafraîchir sur les écrans de suivi et de tableau de bord.
- Feuilles modales (« bottom sheets ») — les actions secondaires (choix du délai, sélection du créneau, du moyen de paiement, détail d'un article) s'ouvrent en feuille remontant du bas de l'écran plutôt qu'en fenêtre pop-up centrée.
- Écran de démarrage (splash screen) et icône d'application — affichés au lancement depuis l'écran d'accueil du téléphone, configurés via le manifeste PWA.
- Respect des zones sûres (« safe areas ») — prise en compte des encoches, barres de statut et boutons de geste des téléphones récents.
- Retour visuel immédiat — état de chargement (squelettes d'écran), retour visuel au toucher sur chaque bouton.
- Notifications push — utilisées pour informer le client de chaque changement de statut de sa commande et pour alerter l'administrateur d'une nouvelle commande ou d'un incident.
- Mode plein écran — une fois installée, l'application s'ouvre sans barre d'adresse ni interface de navigateur.

16.2 Fonctionnement hors ligne — périmètre précis

La PWA distingue clairement ce qui reste utilisable sans connexion de ce qui exige impérativement une connexion active, afin d'éviter toute ambiguïté d'implémentation.

Disponible hors connexion	Nécessite une connexion Internet
Consultation des dernières commandes déjà chargées	Paieement (toujours)
Consultation du dernier statut connu d'une commande	Validation finale d'une commande
Consultation des données déjà mises en cache	Confirmation de paiement
Préparation du panier (ajout/retrait d'articles)	Géolocalisation distante / sélection d'adresse via Google Maps
	Toute synchronisation avec le serveur

Important — un paiement ne doit jamais être placé dans une file d'attente hors ligne. Si la connexion est absente au moment du paiement, l'action est bloquée avec un message explicite invitant le client à retrouver une connexion, plutôt que d'être mise en attente pour un envoi différé.

- La préparation du panier (ajout/retrait d'articles) peut être mise en file d'attente locale et synchronisée au retour du réseau, car elle ne comporte aucune décision financière ou d'engagement irréversible.

16.3 Performance

- Temps de chargement de la page d'accueil inférieur à 2,5 secondes sur connexion mobile 3G/4G.
- Mise à jour du panier et du prix total en moins de 200 ms (calcul côté client pour l'affichage, avec recalcul officiel côté serveur avant paiement — section 6.6).

16.4 Compatibilité

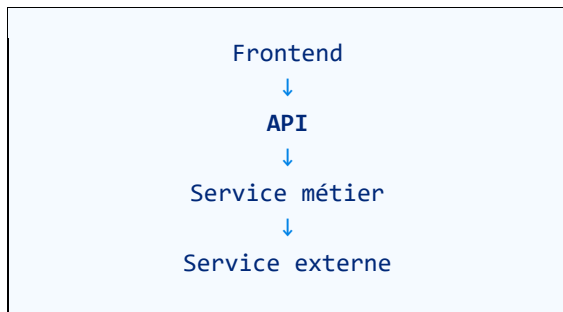
- Conception mobile-first : chaque écran est conçu et testé en priorité dès 360 px de largeur, avant son adaptation aux formats tablette et bureau.
- Support des principaux navigateurs mobiles (Chrome, Safari) et bureau (Chrome, Edge, Firefox).

16.5 Protection des données

Voir le détail complet en section 14.6 ; les exigences de sécurité et de confidentialité sont désormais centralisées dans la section dédiée « Sécurité ».

17. Gestion des erreurs

Une stratégie d'erreur cohérente est appliquée à travers toutes les couches de l'application, afin de garantir un diagnostic rapide et une expérience utilisateur claire même en cas de dysfonctionnement.



- Chaque requête porte un identifiant unique (request id), propagé à travers toutes les couches et inclus dans les logs, pour relier un incident signalé par un utilisateur aux traces techniques correspondantes.
- Les erreurs techniques (ex. service externe indisponible, exception non prévue) sont strictement séparées des erreurs métier (ex. créneau complet, article désactivé) dans le code et dans les réponses API.
- Les messages affichés au client restent compréhensibles et actionnables (« Ce créneau n'est plus disponible, merci d'en choisir un autre »), sans exposer de détails techniques internes.
- Les appels aux services externes (paiement, email, Maps) intègrent une logique de retry avec délai croissant en cas d'échec transitoire.
- Un monitoring applicatif détecte et alerte en cas de hausse anormale du taux d'erreur ou d'indisponibilité d'un service externe critique.

18. Tests & qualité

18.1 Tests unitaires

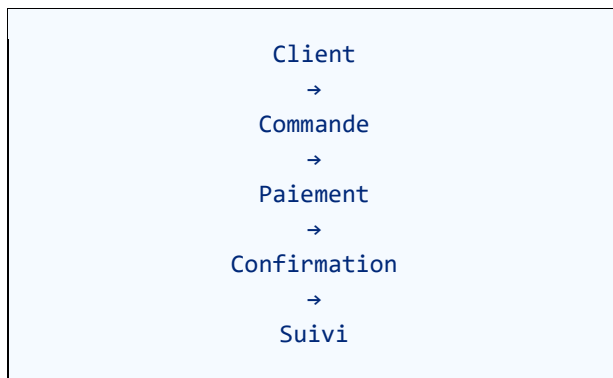
- Calcul du panier (sous-total, quantités, articles hors catalogue)
- Calcul des frais de livraison et de la majoration de délai
- Calcul de la réduction (points de fidélité, abonnement)
- Attribution et consommation des points de fidélité
- Validité des transitions de statut d'une commande (section 3.3)

18.2 Tests API

- Création de commande (cas valide et cas invalides)
- Initiation et confirmation de paiement, y compris les webhooks
- Traitement d'un callback de paiement (y compris callback dupliqué)
- Gestion du catalogue (création, modification de prix, désactivation)
- Authentification (client par OTP, administrateur par identifiants)

18.3 Tests de bout en bout (E2E)

Le parcours nominal complet doit être couvert par un scénario automatisé :



Les scénarios d'échec suivants doivent également être couverts par des tests dédiés (unitaires, API ou E2E selon le cas) :

- Paiement échoué
- Paiement interrompu par le client
- Double callback de paiement
- Double clic sur « Payer »
- Perte de connexion pendant la préparation du panier
- Article désactivé par l'administrateur pendant qu'un client le consulte
- Modification de prix par l'administrateur pendant qu'une commande est en préparation (le prix reste figé pour la commande en cours)
- Commande annulée après paiement réussi (déclenchement du remboursement)
- Erreur ou indisponibilité de l'API Google Maps (repli sur saisie manuelle de l'adresse)
- Échec d'envoi de l'email de confirmation (retry puis alerte interne, sans bloquer la commande)

19. Gestion de projet

19.1 Objectif du MVP

Permettre à Blanchix de recevoir, payer, traiter et livrer ses premières commandes, avec une base technique fiable, sécurisée et prête à évoluer sans refonte.

19.2 Contenu du MVP

- Accueil et vitrine
- Parcours de commande complet (5 étapes), y compris créneaux de collecte/livraison, notes client et article personnalisé
- Catalogue dynamique géré par l'administrateur
- Adresse et géolocalisation (Google Maps)
- Paiement Orange Money / MTN Mobile Money avec workflow serveur complet (section 6)

- Confirmation de compte et de commande par email, reçu PDF
- Cycle de vie complet de la commande avec historique des statuts (section 3)
- Suivi de commande côté client
- Tableau de bord administrateur : commandes, catalogue, créneaux, incidents (section 12)
- Gestion des incidents (section 10)
- Sécurité de base : validation serveur, rate limiting, protection des routes, recalcul serveur des prix (section 14)
- Sauvegardes MongoDB quotidiennes + snapshots VPS + backup externe (section 15.3)
- PWA mobile-first : installation, tab bar, cache offline limité à la consultation (section 16)

19.3 Contenu de la V1

- Paiement par carte bancaire
- Programme de fidélité (points, section 2.6)
- Notifications push (client et administrateur, en complément de l'email)
- Statistiques améliorées du tableau de bord
- Renforcement du cache offline PWA

19.4 Contenu de la V2

- Abonnements Basic/Silver/Gold, une fois les règles métier validées (section 2.7)
- Interface dédiée au livreur (consultation et mise à jour des statuts logistiques)
- Confirmation de compte et de commande via WhatsApp (lien wa.me puis WhatsApp Business API)
- Automatisation logistique avancée (optimisation des tournées, notifications enrichies)
- Statistiques avancées et export de données

19.5 Livrables attendus

- Code source versionné (dépôt Git) avec documentation d'installation.
- Environnements de démonstration (staging) et de production.
- Documentation des endpoints API (backend Next.js).
- Charte graphique appliquée à l'ensemble des écrans (référence : charte graphique Blanchix v1).
- Guide d'utilisation du tableau de bord administrateur.
- Procédure de sauvegarde et de restauration documentée et testée (section 15.3).
- Suite de tests automatisés (unitaires, API, E2E — section 18).

19.6 Critères d'acceptation

Parcours nominal

- Un client peut passer une commande complète (informations → adresse/créneau → panier → délai/créneau → paiement) sans blocage, sur mobile comme sur ordinateur.

- Le prix total se met à jour instantanément à chaque modification du panier ou du délai choisi, et est recalculé à l'identique côté serveur avant tout paiement.
- Un reçu PDF conforme est généré automatiquement après chaque paiement confirmé côté serveur.
- Un email de confirmation exact (numéro de commande, nom, montant, reçu PDF joint) est envoyé automatiquement au client après chaque paiement validé — canal WhatsApp prévu à partir de la V2.
- Un email de bienvenue confirmant la création du compte est envoyé automatiquement dès la première commande — canal WhatsApp prévu à partir de la V2.
- Le tableau de bord administrateur reflète en temps réel (ou quasi temps réel) les commandes, le chiffre d'affaires, les collectes prévues et les incidents ouverts.
- L'application est installable en PWA et reste consultable (lecture du dernier statut) en l'absence de connexion, sans jamais permettre de payer hors ligne.
- L'administrateur peut créer, modifier le prix et désactiver un article du catalogue sans intervention technique, et ce changement se reflète immédiatement dans le panier client, sans affecter les commandes déjà passées.
- Sur mobile, l'application se navigue via une tab bar basse fixe et reproduit les gestes et transitions attendus d'une application native.
- Chaque transition de statut d'une commande est enregistrée dans son historique (statusHistory) avec la date, l'auteur et un commentaire éventuel.

Cas d'échec et cas limites

- Un paiement échoué replace la commande en PAIEMENT_ECHOUÉ et propose au client de réessayer, sans double facturation.
- Un paiement interrompu par le client (fermeture de l'application avant callback) n'engendre ni double transaction ni commande fantôme confirmée par erreur.
- Un callback de paiement reçu plusieurs fois pour la même transaction ne produit qu'un seul effet métier (idempotence vérifiée par test, section 18.2).
- Un double clic sur « Payer » ne crée pas deux transactions distinctes.
- Une perte de connexion pendant la préparation du panier n'entraîne pas de perte de données ; la synchronisation reprend automatiquement au retour du réseau.
- Un article désactivé par l'administrateur pendant qu'un client compose son panier est retiré proprement du panier côté client, avec message explicite, sans erreur bloquante.
- Une modification de prix par l'administrateur pendant qu'une commande est en cours de préparation n'affecte pas le prix déjà affiché au client tant que la commande n'est pas validée à nouveau.
- Une commande annulée après paiement réussi déclenche un remboursement traçable (statut REMBOURSEE, entrée dans payments et audit_logs).
- Une indisponibilité de l'API Google Maps propose un repli en saisie manuelle de l'adresse, sans bloquer la commande.

- Un échec d'envoi d'email de confirmation déclenche un retry automatique puis une alerte interne, sans jamais bloquer la progression de la commande elle-même.

20. Tarification & budget du projet

Cette section présente une estimation budgétaire indicative couvrant le développement de la plateforme — enrichi du périmètre technique renforcé (cycle de vie de commande, paiement sécurisé, logistique, incidents, audit, tests) — et son fonctionnement récurrent. Les montants sont donnés à titre de cadrage et devront être confirmés avec le prestataire retenu à l'issue du chiffrage détaillé.

20.1 Méthodologie de chiffrage

- Développement facturé au forfait par module fonctionnel, sur la base du périmètre décrit en section 2, ou en régie à un taux journalier moyen (TJM) si le client préfère un pilotage à l'avancement.
- Les coûts d'infrastructure (section 15) et les frais de transaction (mobile money, carte) sont récurrents et distincts du coût de développement.
- Toute évolution du périmètre en cours de projet (nouvelle fonctionnalité non listée) fait l'objet d'un avenant chiffré séparément.

20.2 Coût de développement par module (MVP)

Module	Contenu	Estimation (FCFA)
Vitrine & parcours découverte	Accueil, page « Comment ça fonctionne », charte graphique intégrée	250 000 – 400 000
Parcours de commande & créneaux	5 étapes, créneaux de collecte/livraison, notes client, article personnalisé	850 000 – 1 300 000
Cycle de vie & moteur de statuts	Machine à états de la commande, statusHistory, transitions, incidents	500 000 – 800 000
Paiement sécurisé	Orange Money / MTN MoMo / carte, idempotence, webhooks, gestion des échecs	700 000 – 1 100 000
Catalogue & back-office	Gestion des articles/prix par l'administrateur, tableau de bord complet (section 12)	550 000 – 850 000
Authentification & rôles	NextAuth (client par OTP, admin par identifiants), middleware de protection	200 000 – 350 000
Notifications & audit	Emails transactionnels, journal des envois, journal d'audit des actions sensibles	300 000 – 500 000
PWA & expérience mobile-first	Manifest, service worker, cache offline, tab bar, notifications push	350 000 – 550 000
Sécurité & tests automatisés	Validation Zod, rate limiting, tests unitaires/API/E2E	350 000 – 600 000

Module	Contenu	Estimation (FCFA)
Déploiement & mise en production	Déploiement sur le VPS, sauvegardes, documentation	200 000 – 350 000

	Total développement MVP (estimation)
FCFA	4 250 000 – 6 800 000 FCFA (≈ 6 500 – 10 500 €)

Fourchette large car dépendante du niveau de finition demandé sur l'expérience mobile-first, du prestataire de paiement finalement retenu et du nombre d'allers-retours de recette.

20.3 Coût de développement des évolutions V1 et V2

Phase	Contenu	Estimation (FCFA)
V1	Paieement carte bancaire, programme de fidélité, notifications push, statistiques améliorées	600 000 – 950 000
V2	Abonnements Basic/Silver/Gold, confirmation WhatsApp, interface livreur, automatisation logistique, statistiques avancées	1 000 000 – 1 700 000

20.4 Coûts récurrents mensuels

Poste	Détail	Coût estimé
Hébergement VPS Contabo	Cloud VPS 6 (voir section 15.1)	6,00 € / mois (24 premiers mois), puis 7,50 €
Nom de domaine	Renouvellement annuel ramené au mois	≈ 1 à 2 € / mois
Service d'envoi d'email transactionnel	Confirmation de compte/commande, reçus, notifications (volume MVP)	0 à 15 € / mois selon le volume d'envois
API Google Maps / Places	Sélection d'adresse et géolocalisation	Généralement 0 € / mois en usage MVP (quota gratuit), au-delà facturation à l'usage
Certificat SSL	Let's Encrypt	0 €
Stockage de sauvegarde externe	Backup MongoDB quotidien + rétention (section 15.3)	2 à 5 € / mois

	Total récurrent estimé (hors frais de transaction)
FCFA	≈ 9 à 30 € / mois selon le volume d'activité (≈ 6 000 à 19 700 FCFA / mois)

20.5 Frais de transaction (variables selon le volume de commandes)

Moyen de paiement	Frais indicatifs	Remarque
Orange Money	≈ 1 à 2 % du montant de la transaction	Barème contractuel à confirmer avec Orange lors de l'ouverture du compte marchand

Moyen de paiement	Frais indicatifs	Remarque
MTN Mobile Money	≈ 1 à 2 % du montant de la transaction	Barème contractuel à confirmer avec MTN lors de l'ouverture du compte marchand
Carte bancaire (passerelle tierce)	≈ 1,4 à 3,8 % du montant de la transaction	Variable selon le prestataire choisi — voir comparatif section 6.5

Ces frais sont prélevés par les opérateurs de paiement, non par Blanchix ni par le prestataire technique ; ils sont mentionnés ici pour une vision budgétaire complète.

20.6 Maintenance & support post-lancement

- Forfait de maintenance corrective (correction de bugs, mises à jour de sécurité, supervision des sauvegardes) : recommandé à partir de 60 000 – 120 000 FCFA / mois selon le niveau de réactivité attendu.
- Évolutions fonctionnelles hors forfait : facturées au forfait ou en régie, sur devis, comme en section 20.3.
- Supervision de l'infrastructure (charge VPS, sauvegardes, renouvellement des certificats, journal d'audit) : incluse dans le forfait de maintenance ou en option séparée.

20.7 Récapitulatif budgétaire global

Poste	Nature	Montant estimé
Développement MVP	Ponctuel	4 250 000 – 6 800 000 FCFA
Infrastructure & services tiers	Récurrent mensuel	≈ 6 000 – 19 700 FCFA / mois
Maintenance corrective	Récurrent mensuel (recommandé)	60 000 – 120 000 FCFA / mois
Frais de transaction	Variable, au prorata des ventes	1 % à 3,8 % du volume encaissé
Évolutions V1	Ponctuel, à programmer après le MVP	600 000 – 950 000 FCFA
Évolutions V2	Ponctuel, à programmer après la V1	1 000 000 – 1 700 000 FCFA

Cette vue d'ensemble doit être validée et affinée avec le prestataire retenu avant contractualisation ; elle sert de base de discussion budgétaire, non de devis ferme.

21. Annexes

21.1 Arborescence Next.js indicative

- /app — pages publiques (accueil, comment-ca-marche, commander/[étape], suivi/[numero])
- /app/admin — espace administrateur : commandes, catalogue, créneaux, incidents, audit, statistiques
- /app/api — Route Handlers (orders, payments, webhooks/paiement, users, catalog, slots, incidents, notifications, auth)
- /lib — clients Axios, configuration React Query, connexion Mongoose, schémas Zod

- /models — schémas Mongoose (User, Order, CatalogItem, Payment, Subscription, Incident, Notification, DeliverySlot, AuditLog)
- /services — logique métier isolée (calcul de prix, moteur de statuts, idempotence de paiement, fidélité)
- /public — assets statiques, manifest.json et icônes de la PWA

21.2 Glossaire

Terme	Définition
PWA	Progressive Web App — application web installable, fonctionnant hors-ligne partiellement
FCFA	Franco CFA, devise utilisée pour l'ensemble des montants
OTP	One Time Password — code à usage unique envoyé par SMS pour l'authentification
SSR	Server-Side Rendering — rendu des pages côté serveur (Next.js)
Idempotence	Propriété garantissant qu'une même opération, exécutée plusieurs fois, produit le même résultat qu'exécutée une seule fois — essentiel pour les paiements
Webhook / callback	Requête envoyée automatiquement par un service externe (opérateur de paiement) pour notifier un événement (ex. paiement confirmé)
RBAC	Role-Based Access Control — modèle de permissions fondé sur des rôles (client, admin, livreur...)
Snapshot	Copie figée d'une donnée à un instant donné (ex. prix d'un article au moment de la commande), non recalculée rétroactivement
Audit log	Journal immuable des actions sensibles effectuées sur le système, à des fins de traçabilité
statusHistory	Historique complet et immuable des statuts successifs d'une commande

21.3 Référentiel des questions clés

Ce tableau relie chaque question structurante posée en cadrage de projet à la section du présent document qui y répond.

Question	Section de référence
Que peut faire le client ?	Section 2 — Périmètre fonctionnel
Que peut faire l'administrateur ?	Section 12 — Tableau de bord administrateur
Que peut faire le livreur ?	Section 9.5 — Assignment d'un livreur
Quel est le cycle complet d'une commande ?	Section 3 — Cycle de vie d'une commande
Comment fonctionne exactement le paiement ?	Section 6 — Paiement, module renforcé
Que se passe-t-il lorsqu'un paiement échoue ?	Section 6.4 — Gestion des cas d'échec
Comment fonctionne la collecte ?	Section 9.1 et 9.4 — Créneaux et modèle logistique

Question	Section de référence
Comment fonctionne la livraison ?	Section 9.2 et 9.4 — Créneaux et modèle logistique
Comment gérer un article sans prix ?	Section 2.4 (étape 3bis) — Article personnalisé
Comment gérer les incidents ?	Section 10 — Gestion des incidents
Comment sont calculés les prix ?	Section 2.4 (étape 3) et 6.6 — Recalcul serveur
Comment sont gérés les points de fidélité ?	Section 2.6 — Programme de fidélité
Comment sont gérés les abonnements ?	Section 2.7 — Abonnements (V2)
Quelles notifications sont envoyées ?	Section 11 — Notifications
Quelles données sont stockées ?	Section 5 — Modèle de données
Qui peut modifier quoi ?	Section 7.2 — Rôles et permissions
Comment les actions sont-elles auditées ?	Section 13 — Audit et traçabilité
Que fonctionne hors ligne ?	Section 16.2 — Fonctionnement hors ligne
Comment les données sont-elles sauvegardées ?	Section 15.3 — Stratégie de sauvegarde
Comment le système est-il testé ?	Section 18 — Tests et qualité